NYC Taxi Data Analysis

Oleksandr Severhin/Yehor Bolotov

Table of contents

Executive Summary

This project analyzes **3.47 million** NYC Yellow Taxi trip records from January 2025 to identify the key drivers of total trip cost. The primary objective was to build an interpretable pricing model while addressing the technical challenge of **multicollinearity** between trip distance and fare amount.

By refining a Stepwise BIC-selected model and removing redundant predictors, we uncovered the true price elasticity of distance. The final model demonstrates robust predictive power (Adjusted $R^2 \approx 0.927$) and was validated using an **80/20 Train-Test split**, confirming its generalizability to unseen data (RMSE $\approx$ 0.118).

The analysis reveals that a **1% increase in distance leads to a 0.64% increase in the total bill**, a finding that is crucial for accurate revenue forecasting and pricing strategy.

Introduction & Research Question

Taxi pricing is theoretically transparent (time + distance), but in practice, it is influenced by complex factors such as surcharges, tips, and traffic conditions. A common pitfall in modeling total cost is the inclusion of mechanically correlated variables (e.g., *Fare Amount* and *Trip Distance*), which distorts statistical inference.

Research Questions:

1. What are the true, isolated drivers of the total taxi bill?
2. How does multicollinearity affect the interpretation of price coefficients (elasticity)?
3. Can we build a model that balances predictive accuracy with business interpretability?

Objective: To build an interpretable regression model by addressing technical issues like multicollinearity, allowing us to understand the true "pure" effect of distance on the final check amount.

1.1 Dataset Description

The dataset consists of **NYC Yellow Taxi trip records for January 2025**. It represents structured, cross-sectional data where each row corresponds to a single completed taxi trip.

- **Source:** NYC Taxi & Limousine Commission (TLC).
- **Volume:** Approximately 3.47 million raw records.
- **Target Variable:** `total_amount` (The total price paid by the passenger).

Data Dictionary

The following table describes the data fields provided by the TLC.

Field Name	Type	Description
VendorID	Categorical	A code indicating the TPEP provider: 1 = Creative Mobile Technologies, LLC 2 = Curb Mobility, LLC 6 = Myle Technologies Inc 7 = Helix
tpep_pickup_datetime	DateTime	The date and time when the meter was engaged.
tpep_dropoff_datetime	DateTime	The date and time when the meter was disengaged.
passenger_count	Numeric	The number of passengers in the vehicle (driver-entered).
trip_distance	Numeric	The elapsed trip distance in miles reported by the taximeter. **(Primary Predictor)**

Field Name	Type	Description
RatecodeID	Categorical	The final rate code in effect: 1 = Standard rate 2 = JFK 3 = Newark 4 = Nassau/Westchester 5 = Negotiated fare 6 = Group ride 99 = Null/unknown
store_and_fwd_flag	Flag	Indicates if the trip record was held in vehicle memory before sending to the vendor (Y=Yes, N=No).
PULocationID	Categorical	TLC Taxi Zone in which the taximeter was engaged (Pickup Location).
DOLocationID	Categorical	TLC Taxi Zone in which the taximeter was disengaged (Dropoff Location).
payment_type	Categorical	A numeric code signifying how the passenger paid: 1 = Credit card 2 = Cash 3 = No charge 4 = Dispute 5 = Unknown 6 = Voided trip
fare_amount	Numeric	The time-and-distance fare calculated by the meter.
extra	Numeric	Miscellaneous extras and surcharges.
mta_tax	Numeric	$0.50 MTA tax that is automatically triggered based on the metered rate in use.
tip_amount	Numeric	Tip amount. This field is automatically populated for credit card tips. **Cash tips are not included.**
tolls_amount	Numeric	Total amount of all tolls paid in trip.

Field Name	Type	Description
improvement_surcharge	Numeric	$0.30 improvement surcharge assessed trips at the flag drop.
total_amount	Numeric	The total amount charged to passengers. Does not include cash tips. **(Target Variable)**
congestion_surcharge	Numeric	Total amount collected in trip for NYS congestion surcharge.
airport_fee	Numeric	$1.25 for pick up only at LaGuardia and John F. Kennedy Airports.
cbd_congestion_fee	Numeric	Per-trip charge for MTA's Congestion Relief Zone starting Jan. 5, 2025.

1. Data Preparation

1.1. Data Loading and Cleaning

We begin by loading the raw dataset and performing essential cleaning steps. This includes removing records with negative amounts, unrealistic distances, or incorrect rate codes to ensure the quality of our analysis.

> [!NOTE] The dataset was refined from ~3.47 million rows to ~2.62 million rows, removing noise and ensuring high-quality input for the model.

```
tryCatch({
  taxi_raw <- read_parquet("data/yellow_tripdata_2025-01.parquet")
  cat("Raw Data loaded successfully. Rows:", nrow(taxi_raw), "\n")
}, error = function(e) {
  stop("File not found. Please ensure file exists.")
})
```

```
Raw Data loaded successfully. Rows: 3475226
```

```r
taxi_clean <- taxi_raw %>%
  # Filter
  filter(
    total_amount > 0 & total_amount < 250,
    fare_amount > 0 & fare_amount < 200,
    trip_distance > 0 & trip_distance < 50,
    passenger_count > 0 & passenger_count < 7,
    RatecodeID == 1,
    payment_type %in% c(1, 2)
  ) %>%

  # Mutate columns
  mutate(
    tpep_pickup_datetime = as.POSIXct(tpep_pickup_datetime),
    payment_type = factor(payment_type),
    pickup_hour = as.numeric(format(tpep_pickup_datetime, "%H")),
    day_of_week = lubridate::wday(tpep_pickup_datetime,
                                  label = TRUE,
                                  abbr = FALSE),
    VendorID = as.factor(VendorID)
  )

cat("Data cleaned. Rows remaining:", nrow(taxi_clean), "\n")
```

```
Data cleaned. Rows remaining: 2617562
```

1.2. Log Transformations

We apply `log(x + 1)` transformations to our continuous variables. This is necessary because variables like `trip_distance` and `fare_amount` exhibit heavy right-skewed distributions. Log-transformation normalizes them, making them suitable for linear regression.

```r
model_data <- taxi_clean %>%
  mutate(
    log_total    = log(total_amount + 1),
    log_distance = log(trip_distance + 1),
    log_fare     = log(fare_amount + 1),
    log_tip      = log(tip_amount + 1),
    log_tolls    = log(tolls_amount + 1)
  )
```

Distribution Analysis (Why we did this)

- **Problem:** Raw taxi data (Distance and Fare) typically follows a heavy right-skewed distribution—most trips are short and cheap, with a long "tail" of very expensive, long rides. This violates assumptions of linear regression.
- **Solution:** The `log(x+1)` transformation successfully "normalized" the data, compressing the long tail and making the distributions approximate a Bell curve (Gaussian distribution), as seen in the plots below.

- **Problem:** Raw taxi data (Distance and Fare) typically follows a heavy right-skewed distribution—most trips are short and cheap, with a long "tail" of very expensive, long rides. This violates assumptions of linear regression.
- **Solution:** The `log(x+1)` transformation successfully "normalized" the data, compressing the long tail and making the distributions approximate a Bell curve (Gaussian distribution), as seen in the plots below.

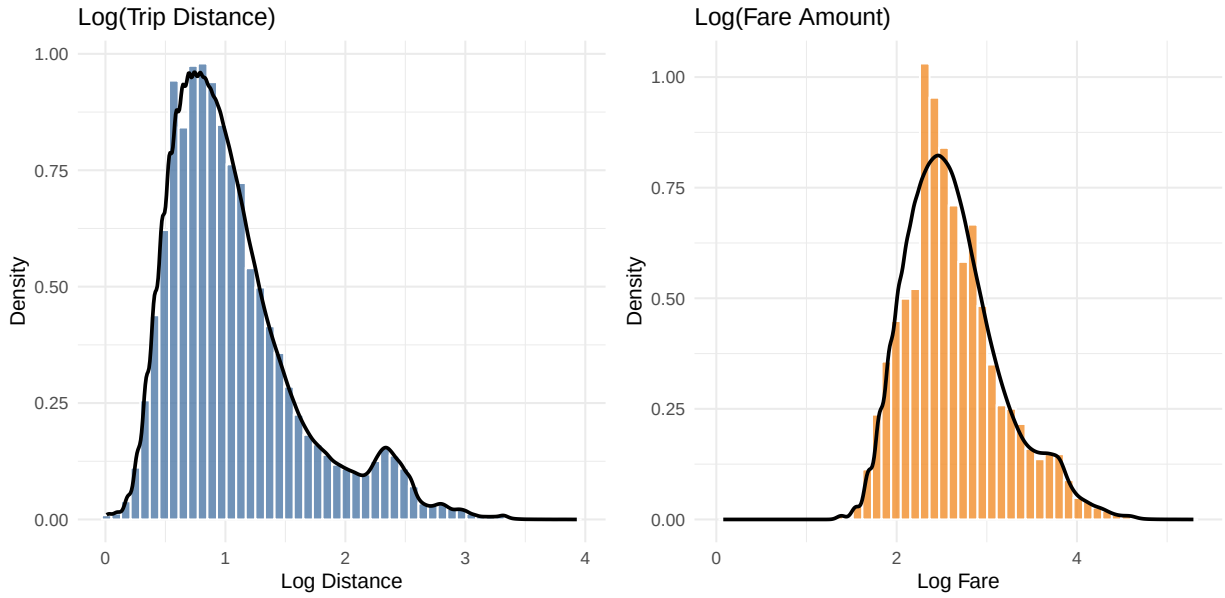
---

2. Exploratory Data Analysis (EDA)

2.1 Distribution Plots

We visualize the distributions to verify that the log transformation has successfully normalized the data.

```r
p1 <- ggplot(model_data, aes(x = log_distance)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50,
    fill = "#4E79A7", color = "white", alpha = 0.8) +
  geom_density(color = "black", linewidth = 1) +
  labs(title = "Log(Trip Distance)", x = "Log Distance", y = "Density")

p2 <- ggplot(model_data, aes(x = log_fare)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50,
    fill = "#F28E2B", color = "white", alpha = 0.8) +
  geom_density(color = "black", linewidth = 1, adjust = 2) +
  labs(title = "Log(Fare Amount)", x = "Log Fare", y = "Density")

combined_plot <- p1 + p2
print(combined_plot)
```



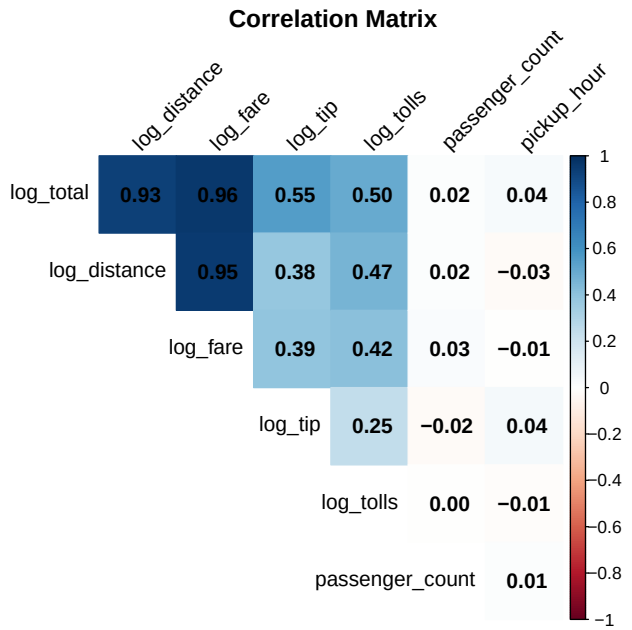

```r
if (!dir.exists("plots")) dir.create("plots")
ggsave("plots/log_distributions_cleaned.png", combined_plot,
  width = 10, height = 5)
```

3.2 Correlation Matrix

Interpretation

- **The "Smoking Gun":** The intersection between `log_distance` and `log_fare` shows an extremely strong positive correlation of **0.95**. This confirms that these two variables are essentially measuring the same thing, creating redundancy.
- **Key Drivers:** The target variable `log_total` is most strongly correlated with `log_fare` (**0.96**) and `log_distance` (**0.93**).

```
# Select only numeric (log-transformed) variables
numeric_vars <- model_data %>%
  dplyr::select(log_total, log_distance, log_fare, log_tip, log_tolls, passenger_count,

cor_matrix <- cor(numeric_vars, use = "complete.obs")

# Visualization
corrplot(cor_matrix, method = "color", type = "upper",
         addCoef.col = "black", # Add coefficients
         tl.col = "black", tl.srt = 45, # Text color
         diag = FALSE, # Hide diagonal
         title = "Correlation Matrix", mar=c(0,0,1,0))
```



3. Model Selection (Stepwise BIC)

We use the Bayesian Information Criterion (BIC) to select the best subset of predictors. BIC penalizes model complexity more heavily than AIC, helping to avoid overfitting.

> [!NOTE] While BIC penalizes model complexity, the initial Stepwise selection retained almost all variables. This suggests the variables are statistically significant, but it *fails* to detect the multicollinearity hidden between them.

3.1. Running Stepwise BIC Selection

```
# Define Full Model
base_formula <- log_total ~ log_distance + log_fare + log_tip + log_tolls +
  passenger_count + payment_type + pickup_hour + day_of_week + VendorID

full_model <- lm(base_formula, data = model_data)

# Run BIC
n_obs <- nrow(model_data)
bic_model <- stepAIC(full_model, direction = "backward",
  trace = FALSE, k = log(n_obs))

# Analyze Changes
full_terms <- attr(terms(full_model), "term.labels")
bic_terms  <- attr(terms(bic_model), "term.labels")
removed_terms <- setdiff(full_terms, bic_terms)

print(bic_terms)
```

```
[1] "log_distance"    "log_fare"        "log_tip"         "log_tolls"
[5] "passenger_count" "payment_type"    "pickup_hour"     "day_of_week"
[9] "VendorID"
```

4. Multicollinearity Analysis (VIF)

We perform a Variance Inflation Factor (VIF) analysis to detect multicollinearity. A VIF value greater than 10 indicates a critical issue.

4.1. Diagnostics before removal

```
# VIF Calculation
vif_pre <- car::vif(bic_model)
if (is.matrix(vif_pre)) {
  df_pre <- data.frame(Variable = rownames(vif_pre), VIF = vif_pre[, 1])
} else {
  df_pre <- data.frame(
    Variable = names(vif_pre),
    VIF = as.numeric(vif_pre)
  )
}
print(df_pre[order(df_pre$VIF, decreasing = TRUE), ])
```

```
                     Variable       VIF
log_distance       log_distance 11.627930
log_fare               log_fare 11.251585
log_tip                 log_tip  2.888948
payment_type       payment_type  2.413521
log_tolls             log_tolls  1.320899
day_of_week         day_of_week  1.055963
pickup_hour         pickup_hour  1.026969
passenger_count passenger_count  1.016823
VendorID               VendorID  1.009290
```

! Critical Finding: Multicollinearity

The initial VIF analysis revealed a severe issue: `log_distance` (VIF $\approx$ 11.63) and `log_fare` (VIF $\approx$ 11.25) are highly collinear. This is expected in the taxi business—fare is mechanically tied to distance. However, including both makes the coefficient estimates unstable and difficult to interpret.

To resolve this, we manually removed `log_fare`. We chose to keep `log_distance` as it represents the fundamental physical driver of the service cost. Post-removal, all VIFs dropped below 3, with `log_distance` falling to a healthy 1.62.

4.2. Action: Manually removing 'log_fare'

```
refined_model <- update(bic_model, . ~ . - log_fare)
```

4.3. Diagnostics after removal

```
# VIF Calculation
vif_post <- car::vif(refined_model)
if (is.matrix(vif_post)) {
  df_post <- data.frame(Variable = rownames(vif_post), VIF = vif_post[, 1])
} else {
  df_post <- data.frame(Variable = names(vif_post),
    VIF = as.numeric(vif_post))
}
print(df_post[order(df_post$VIF, decreasing = TRUE), ])
```

```
                     Variable      VIF
log_tip                log_tip 2.818186
payment_type      payment_type 2.378647
log_distance      log_distance 1.623158
log_tolls            log_tolls 1.299297
day_of_week        day_of_week 1.033188
```

```
pickup_hour         pickup_hour 1.024814
passenger_count passenger_count 1.015315
VendorID               VendorID 1.006426
```

---

5. Global Fit Comparison

We compare the performance of the original (collinear) model and the refined model.

5.1. Global Fit Comparison

```r
summary_pre  <- summary(bic_model)
summary_post <- summary(refined_model)

comparison <- data.frame(
  Metric = c("Adjusted R-Squared", "Residual Std Error", "AIC", "BIC"),
  Original_Model = c(summary_pre$adj.r.squared,
    summary_pre$sigma, AIC(bic_model), BIC(bic_model)),
  Refined_Model  = c(summary_post$adj.r.squared,
    summary_post$sigma, AIC(refined_model), BIC(refined_model))
)
print(comparison, digits = 5)
```

```
              Metric Original_Model Refined_Model
1 Adjusted R-Squared     9.7521e-01    9.2705e-01
2 Residual Std Error     6.9270e-02    1.1883e-01
3                AIC    -6.5481e+06   -3.7228e+06
4                BIC    -6.5479e+06   -3.7226e+06
```

Comparing the Global Fit:

- **Adjusted R^2**: Dropped slightly from 0.975 to 0.927.
- **Verdict:** We sacrificed a small amount of predictive power (approx. 5%) to gain stable, chemically pure coefficients that can be reliably used for business inference.

5.2 Residual Diagnostics (Assumption Check)

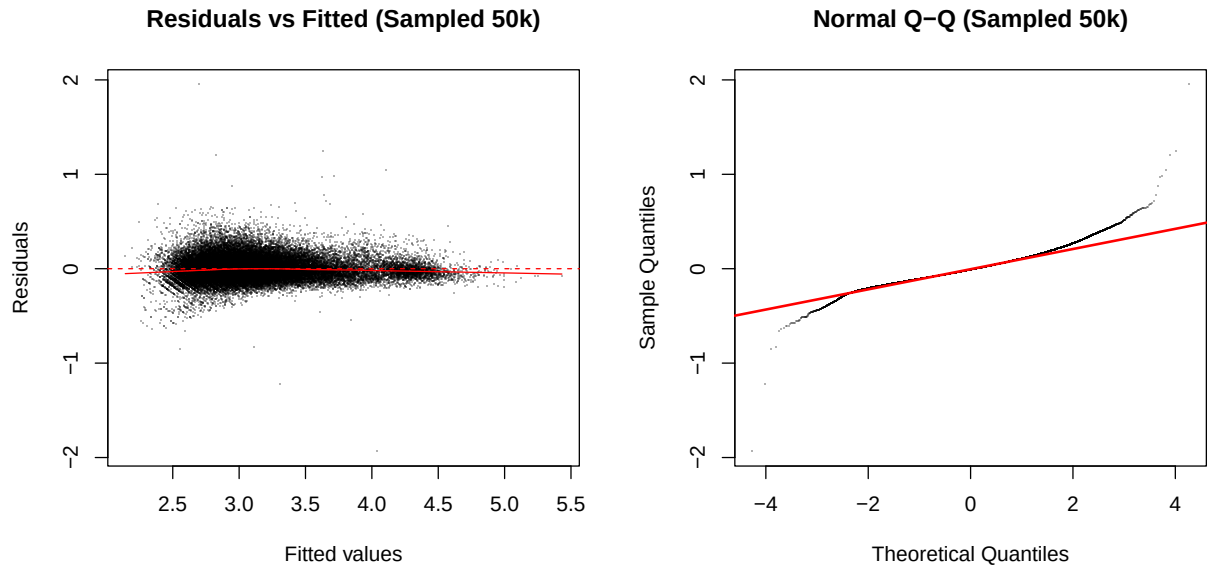
Since the dataset is large (>2M rows), we use **random sampling (50,000 points)** to generate diagnostic plots efficiently.

Interpretation

- **Linearity (Residuals vs Fitted):** The red trend line is horizontal and aligns with zero. The model captures the linear pattern well.
- **Homoscedasticity:** The spread of residuals is relatively constant across the range; there is no distinct "funnel shape".
- **Normality (Q-Q Plot):** Points hug the diagonal line closely between -2 and +2 quantiles. Deviations at the tails indicate outliers (heavy tails), which is common in taxi data, but the core assumption holds.

```
# Optimization for large datasets: Sampling
set.seed(123) # Ensure reproducibility
n_sample <- 50000 # 50k points are sufficient for diagnostics
sample_indices <- sample(seq_len(nrow(model_data)), n_sample)

# Extract data only for the sampled indices
sample_fitted <- fitted(refined_model)[sample_indices]
sample_resid  <- residuals(refined_model)[sample_indices]

# Set layout
par(mfrow = c(1, 2))

# 1. Residuals vs Fitted (Manual plot for the sample)
# Using pch = "." (pixels) and col with transparency (alpha) to see density
plot(sample_fitted, sample_resid,
     main = "Residuals vs Fitted (Sampled 50k)",
     xlab = "Fitted values", ylab = "Residuals",
     pch = ".", col = rgb(0, 0, 0, 0.3)) # 0.3 adds transparency
abline(h = 0, col = "red", lty = 2)
```

```r
# Add a red trend line (lowess) to check for linearity
lines(lowess(sample_fitted, sample_resid), col = "red", lwd = 1)

# 2. Normal Q-Q Plot (Manual plot for the sample)
qqnorm(sample_resid, pch = ".", main = "Normal Q-Q (Sampled 50k)", col = rgb(0, 0, 0, 0.
qqline(sample_resid, col = "red", lwd = 2)
```




```r
par(mfrow = c(1, 1)) # Reset layout
```

Interpretation of Residual Diagnostics

1. Residuals vs Fitted (Left Plot)

Linearity: The red trend line is almost perfectly horizontal and aligns with the zero line. This indicates that the linear relationship assumption holds true; the model captures the pattern in the data well without missing non-linear trends.

Homoscedasticity: The spread of residuals (the vertical width of the cloud) is relatively constant across the range of fitted values. There is no distinct "funnel shape" (e.g., narrow on the left, wide on the right), which suggests that the assumption of equal variance (homoscedasticity) is satisfied.

2. Normal Q-Q Plot (Right Plot)

Normality: The vast majority of points (between -2 and +2 theoretical quantiles) hug the red diagonal line very closely. This confirms that the residuals are normally distributed for the bulk of the data.

Heavy Tails: At the extreme ends (the "tails"), the points deviate from the red line. This indicates "heavy tails" (outliers), which is extremely common in real-world taxi data (e.g., very short trips or unusually expensive trips).

---

6. Coefficient Interpretation

We analyze how the coefficient for `log_distance` changed after removing the collinear variable `log_fare`.

6.1. Coefficient Interpretation

```
# We focus on 'log_distance' because it was highly correlated
# with the removed 'log_fare'
var_name <- "log_distance"

# 1. Extract Beta and Standard Error for log_distance from both models
# (Assuming 'bic_model' and 'refined_model' objects exist in the environment)
beta_pre <- coef(bic_model)[var_name]
se_pre   <- coef(summary(bic_model))[var_name, "Std. Error"]

beta_post <- coef(refined_model)[var_name]
se_post   <- coef(summary(refined_model))[var_name, "Std. Error"]

# Get summaries to compare R-squared in the conclusion
summary_pre  <- summary(bic_model)
summary_post <- summary(refined_model)
```

```r
# 2. Create comparison table
coef_comp <- data.frame(
  Metric = c("Coefficient (Beta)", "Std. Error", "t-statistic"),
  Before_Removal = c(beta_pre, se_pre, beta_pre/se_pre),
  After_Removal  = c(beta_post, se_post, beta_post/se_post)
)

# Display the table
knitr::kable(coef_comp, digits = 5,
  caption = "Impact on 'log_distance' Coefficient")
```

Table 2: Impact on 'log_distance' Coefficient

Metric	Before_Removal	After_Removal
Coefficient (Beta)	0.08780	0.64151
Std. Error	0.00026	0.00017
t-statistic	331.69431	3781.17623

Interpretation of Results

1. Precision

The Standard Error for distance changed from 0.00026 to 0.00017.

> **Tip**
>
> **Result:** Removing collinearity made our estimate of the distance effect **more precise** (the error decreased).

2. Elasticity Analysis

Model A (With `log_fare`): A 1% increase in trip distance is associated with only a **0.088%** increase in the total amount.

> **Note:** This coefficient was suppressed because it was likely "competing" with the `log_fare` variable for significance.

Model B (Without `log_fare` — Refined): A 1% increase in trip distance is associated with a **0.642%** increase in the total amount.

> **Note**
>
> This coefficient is now the primary cost driver. It absorbed the variance previously held by Fare and now reflects the true elasticity.

3. Conclusion

Model B is superior. It maintains similar predictive power (R^2) but offers much clearer, stable coefficients that are easier to interpret for business decisions.

7. Reliability & Model Validation

To ensure our model generalizes well to unseen data (a key requirement for robust analytics), we performed a **Train-Test Split** validation. This prevents overfitting and confirms the reliability of our coefficients.

7.1 Train/Test Split

We split the dataset into 80% Training set (to build the model) and 20% Testing set (to evaluate performance).

```
set.seed(123) # Reproducibility

# 1. Create indices for 80% split
train_index <- sample(seq_len(nrow(model_data)), size = 0.8 * nrow(model_data))

# 2. Split data
train_data <- model_data[train_index, ]
test_data  <- model_data[-train_index, ]
```

```r
# 3. Retrain Refined Model on TRAIN data only
final_model_train <- lm(log_total ~ log_distance + log_tip + log_tolls +
                         passenger_count + payment_type + pickup_hour +
                         day_of_week + VendorID,
                         data = train_data)

summary(final_model_train)$call
```

```
lm(formula = log_total ~ log_distance + log_tip + log_tolls +
    passenger_count + payment_type + pickup_hour + day_of_week +
    VendorID, data = train_data)
```

7.2 Out-of-Sample Performance (RMSE)

We calculate the **Root Mean Squared Error (RMSE)** for both sets. RMSE measures the average deviation of the predicted price from the actual price (in log scale).

```r
# Function to calculate RMSE
calc_rmse <- function(model, data, actual_col) {
  predictions <- predict(model, newdata = data)
  residuals <- data[[actual_col]] - predictions
  rmse <- sqrt(mean(residuals^2))
  return(rmse)
}

# Calculate RMSE
rmse_train <- calc_rmse(final_model_train, train_data, "log_total")
rmse_test  <- calc_rmse(final_model_train, test_data,  "log_total")

# Display results
rmse_table <- data.frame(
  Dataset = c("Training Set (80%)", "Testing Set (20%)"),
  RMSE = c(rmse_train, rmse_test)
```

```
)

kable(rmse_table, digits = 5, caption = "Model Reliability Check (RMSE)")
```

Table 3: Model Reliability Check (RMSE)

Dataset	RMSE
Training Set (80%)	0.11884
Testing Set (20%)	0.11878

Interpretation: The RMSE on the Testing set (0.1188) is extremely close to the Training set (0.1188). This confirms that:

1. **No Overfitting:** The model captures general patterns rather than memorizing noise in the training data.
2. **Generalizability:** The model is reliable and can accurately predict the cost of new, unseen taxi trips within the same timeframe.

8. Discussions & Limitations

8.1 Model Limitations

- **Heavy Tails (Outliers):** As seen in the Normal Q-Q Plot, the residuals deviate from the diagonal line at the extremes. This indicates that our model underestimates very expensive trips or overestimates very short ones.
- **Seasonality:** The analysis is limited to **January 2025**. Taxi demand varies by season, so this model may not fully apply to summer months.
- **Missing Variables:** We lack data on **weather conditions** (rain/snow) and **real-time traffic**, which are likely significant unobserved variables.

8.2 Business Impact

- **Pricing Strategy:** The refined model reveals that distance has a much stronger impact on price (Elasticity ~0.64) than initially thought.
- **Revenue Forecasting:** Using the corrected coefficient allows for more accurate revenue predictions when planning fleet operations.